

■ Crime & Legal Text Classification

Technical Project Report — Multi-Architecture NLP Classification with Keras, and a Streamlit Model-Monitoring Dashboard

Field	Detail
Project type	4-class text classification (NLP) — crime / legal domain
Frameworks	TensorFlow / Keras, scikit-learn, Streamlit, Plotly
Architectures compared	Dense (BoW-style), Conv1D, LSTM, BiLSTM
Dataset size	742 labeled text samples (520 train / 111 val / 111 test)
Report scope	Data pipeline, model design, training setup, evaluation, dashboard

This document is a detailed, engineering-level walkthrough of the **Crime-Legal-Predict** project: a text classifier that assigns a short piece of text to one of four crime/legal categories, together with a production-style Streamlit dashboard built to monitor that model over time. The report covers the dataset, the preprocessing pipeline, all four model architectures that were trained and compared, the full evaluation results, an honest discussion of the results' limitations, and a tour of every dashboard feature.

1. Problem Definition

Given a short free-text description (e.g. an incident report, a complaint, or a case summary), the model predicts which of four categories it belongs to. This is a supervised, single-label, multi-class text classification problem.

Output classes

Label	Class	Samples	Share
0	No crime	377	50.8%
1	Violent crime / drugs	208	28.0%
2	Theft / cybercrime	130	17.5%
3	Corruption / financial crime	28	3.8%

The dataset is **strongly imbalanced**: the majority class ("No crime") makes up roughly half of all samples, while the minority class ("Corruption / financial crime") has only 28 examples in total — this single fact shapes several downstream decisions (class weighting, evaluation caution, dashboard drift checks) described later in this report.

2. Dataset and Train / Validation / Test Split

The raw dataset (`dataset.csv`) contains 742 rows after dropping any rows with missing text or label values. Labels are cast to integers, and the four class labels above are inferred directly from the data.

Split	Samples	Purpose
Train	520 (70%)	Used to fit model weights
Validation	111 (15%)	Used for early stopping / LR scheduling during training

Split	Samples	Purpose
Test	111 (15%)	Held out — used only for final evaluation

The split is performed in two stages with `train_test_split`: first train vs. (val+test) at 70/30, then the remaining 30% is split evenly into validation and test. Both stages use `stratify=label` and a fixed `random_state=42`, so class proportions are preserved across all three splits and the split is fully reproducible.

3. Text Preprocessing

Vectorization

- Text is converted to integer sequences with Keras' `TextVectorization` layer.
- Vocabulary is capped at **5,000 tokens** (`max_tokens=5000`).
- Every sequence is padded/truncated to a fixed length of **32 tokens** (`output_sequence_length=32`).
- The layer lowercases text and strips punctuation by default, then splits on whitespace.
- The vectorizer is fit (`adapt()`) only on the **training** split, to avoid any leakage of validation/test vocabulary into the model.

Because the dataset itself is small, the fitted vocabulary ends up containing only **418 tokens** (including the reserved padding and [UNK] tokens) — far below the 5,000 cap. This fitted vocabulary is persisted to `artifacts/vectorizer_vocab_crime_model.pkl` so the Streamlit app can reload the exact same vectorization at inference time without re-fitting.

Batching pipeline

Each split is wrapped into a `tf.data.Dataset`, batched with `batch_size=16`, and (for the training set) shuffled with a fixed seed. Text is vectorized inside the pipeline via a `.map()` call and prefetched with `AUTOTUNE` for efficient training.

Class imbalance handling

Because "Corruption / financial crime" has very few examples relative to the other classes, `sklearn.utils.class_weight.compute_class_weight("balanced", ...)` is used to compute per-class weights from the training labels. These weights are passed to `model.fit(..., class_weight=...)` so that the loss for minority-class samples counts proportionally more, discouraging the model from simply ignoring rare classes.

4. Model Architectures

Four models were trained and compared. All four share the exact same "head": a 64-dimensional Embedding layer, followed by an architecture-specific encoder, then $\text{Dense}(64, \text{relu}) \rightarrow \text{Dropout}(0.3) \rightarrow \text{Dense}(4, \text{softmax})$. Keeping everything except the encoder identical isolates the effect of *how the text is encoded* on final performance.

Model	Encoder	Design idea
Dense	GlobalAveragePooling1D over token embeddings	Bag-of-words style baseline; averages word meaning, ignores order
Conv1D	Conv1D(64 filters, kernel=3) + GlobalMaxPooling1D	Detects local 3-word patterns / short phrases
LSTM	LSTM(64 units)	Reads the sequence left→right, captures word order
BiLSTM	Bidirectional(LSTM(64 units))	Reads left→right and right→left, fuller context per word

Training configuration (shared across all 4 models)

Setting	Value
Optimizer	Adam, learning rate = 1e-3
Loss function	Sparse Categorical Crossentropy
Batch size	16
Max epochs	30
Early stopping	monitor=val_loss, patience=5, restores best weights
LR schedule	ReduceLROnPlateau — monitor=val_loss, patience=3, factor=0.5, min_lr=1e-6
Class weighting	"balanced" class weights applied during training

5. Evaluation Results (Test Set, 111 samples)

Model	Accuracy	F1 (weighted)	Precision (weighted)	Recall (weighted)
Dense	1.000	1.000	1.000	1.000
Conv1D	0.866	0.871	0.902	0.866
BiLSTM	0.661	0.581	0.727	0.661
LSTM	0.527	0.381	0.549	0.527

Critical interpretation — read this before trusting the numbers above

The Dense model's perfect 1.000 score should **not** be read as "bag-of-words beats recurrent models here." With only 742 total rows, a vocabulary of just 418 tokens, and a 111-row test set (4 of which are from the smallest class), a perfect score is a strong sign of **overfitting / memorization** rather than genuine generalization to unseen text. Conversely, the recurrent models (LSTM, BiLSTM) likely underperformed not because sequence order is unhelpful for this task, but because there simply isn't enough data for them to learn reliable sequential patterns — they are the most parameter-hungry and data-hungry of the four architectures tried. With more data, more regularization, or pretrained embeddings, this ranking could plausibly reverse.

This is precisely why the dashboard (Section 6) includes **Data Drift Detection** and **Error Analysis** tabs — to catch this kind of fragility once the model is exposed to new, more diverse production text, rather than trusting a single static test-set score.

Inference-time behavior

At prediction time, input text is vectorized with the saved vocabulary and passed through the selected model to obtain a softmax probability distribution over the 4 classes. The highest-probability class is returned along with its confidence score.

6. Streamlit Monitoring Dashboard

Beyond the models themselves, the project ships a Streamlit application (`app.py`, ~830 lines) styled as a production ML-monitoring dashboard, organized into seven tabs:

- **Overview** — Test-set Accuracy / Precision / Recall / F1 for the selected model, plus a simulated production-traffic / throughput panel.
- **Confusion Matrix** — Raw and row-normalized confusion matrices, plus a detailed per-class precision / recall / F1 report.
- **Data Drift** — Compares text-length distribution and class-distribution shifts between the reference (training) data and simulated production data using a Kolmogorov–Smirnov (KS) test, with a drift-over-time trend chart.
- **Error Analysis** — Lets you filter predictions down to low-confidence samples, misclassified samples, or both, to inspect exactly where and how the model fails.
- **Alert History** — A simulated daily alert feed flagging accuracy drops, low-confidence spikes, data drift, and traffic anomalies, with color-coded severity badges.
- **Filter & Export** — Filter predictions by class, confidence range, or free-text search, then export the filtered results as CSV or as a summary report.
- **Explainability** — Runs a word-level occlusion test: it re-predicts the input text with one word zeroed out at a time and measures how much the target class's confidence drops, to highlight which words most influenced the model's decision.

The dashboard lets the user switch between all four trained models from the sidebar, so any tab's analysis can be repeated per architecture — useful for comparing not just accuracy, but also drift sensitivity and error patterns across models.

7. Project Structure

```
Crime_Legal_Predict-main/
├── artifacts/
│   ├── model_Dense_crime.keras
│   ├── model_Conv1D_crime.keras
│   ├── model_LSTM_crime.keras
│   ├── model_BiLSTM_crime.keras
│   └── vectorizer_vocab_crime_model.pkl
├── dataset/
│   └── dataset.csv
├── crime_legal_text_classification_keras.ipynb
├── app.py
└── requirements.txt
```

Dependencies

streamlit ≥1.36 · tensorflow ≥2.16 · numpy ≥1.26 · pandas ≥2.0 · scikit-learn ≥1.4 · scipy ≥1.11 · plotly ≥5.20

8. Limitations and Future Work

- **Small dataset (742 rows):** all four models are trained on very limited data; results, especially the Dense model's perfect score, should be treated as preliminary rather than production-grade.
- **Severe class imbalance:** the minority class (28 samples) is under-represented even after class weighting; more data collection for this class would likely help most.

- **Single test-set snapshot:** evaluation relies on one fixed 111-sample split rather than cross-validation, so reported metrics carry meaningful variance.
- **No pretrained embeddings:** embeddings are learned from scratch on a tiny vocabulary (418 tokens); pretrained word vectors could improve the recurrent models substantially.
- **Recommended next steps:** collect more labeled data (especially for the minority class), add k-fold cross-validation, try pretrained embeddings or a lightweight transformer encoder, and use the dashboard's drift/error tools continuously once real production text starts flowing in.

This report was generated to accompany the Crime-Legal-Predict repository and reflects the code and notebook exactly as implemented at the time of writing.